

CSC311H5 Introduction to Machine Learning

Lecture 6

University of Toronto Mississauga

Test 1 Reminder

- Friday Feb 13, 7:15-8:45pm, IB110/IB120
 - ▶ room split will be posted soon
 - ▶ please plan to arrive early if you are able
- Covers materials from weeks 1-4
- Make sure to **bring your own non-programmable calculator**
 - ▶ calculators cannot be shared
- You may bring one single-sided cheat sheet
- See Piazza post for more information

Review

Recall:

- linear models for **classification**
 - ▶ logistic regression
 - ▶ multi-class classification
- limitations of these models: decision boundaries are **linear**, unless feature mapping is used
 - ▶ can we learn more complex models?
 - ▶ or, in other words, can we automatically learn a feature mapping?

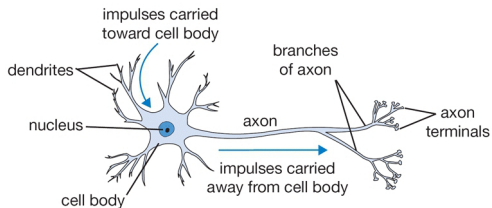
Today:

- neural networks!

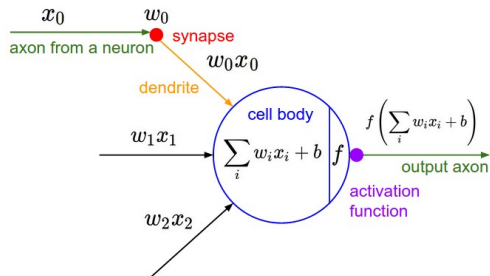
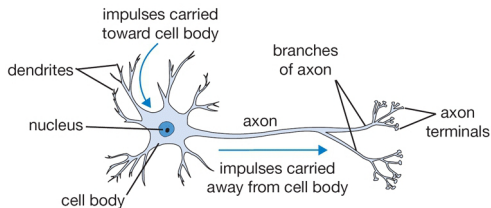
Section 1

Towards Neural Networks

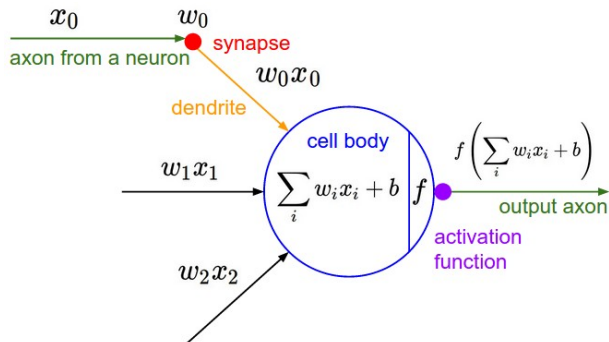
Biological and Artificial Neurons



Biological and Artificial Neurons

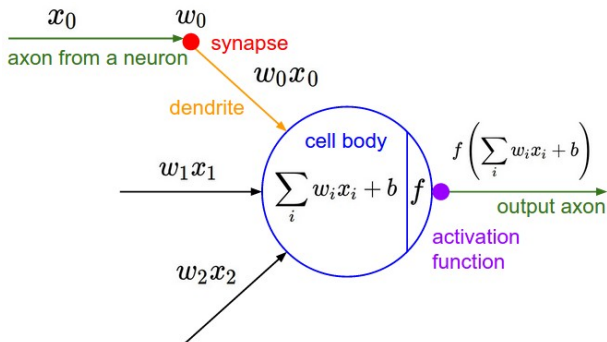


Modeling Individual Neurons



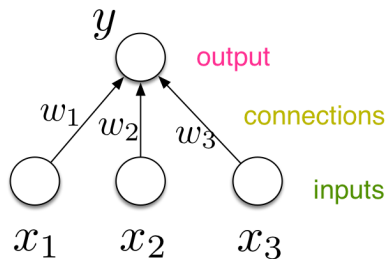
- $x_1, x_2, \dots$: inputs to the neuron
- $w_1, w_2, \dots$: the neuron's **weights**
- b : the neuron's **bias**
- f : an **activation function**
- $y = f(\sum_i x_i w_i + b)$: the neuron's **activation** (output)

Linear Models as a Single Neuron



- $x_1, x_2, \dots$: inputs
- $w_1, w_2, \dots$: components of the **weight vector $\mathbf{w}$**
- b : the **bias**
- f : identity function
- $y = \sum_i x_i w_i + b = \mathbf{w}^T \mathbf{x} + b$

Binary Logistic Regression as a Single Neuron



$$y = \phi(\mathbf{w}^T \mathbf{x} + b)$$

Diagram illustrating the mathematical representation of the single neuron model. The equation is $y = \phi(\mathbf{w}^T \mathbf{x} + b)$. Colored arrows point to the components: a pink arrow points to y (labeled "output"), a blue arrow points to $\mathbf{w}$ (labeled "weights"), a blue arrow points to b (labeled "bias"), a red arrow points to ϕ (labeled "activation function"), and a green arrow points to $\mathbf{x}$ (labeled "inputs").

- $x_1, x_2, \dots$: inputs
- $w_1, w_2, \dots$: components of the **weight vector** $\mathbf{w}$
- b : the **bias**
- $f = \sigma$
- $y = \sigma(\sum_i x_i w_i + b) = \sigma(\mathbf{w}^T \mathbf{x} + b)$

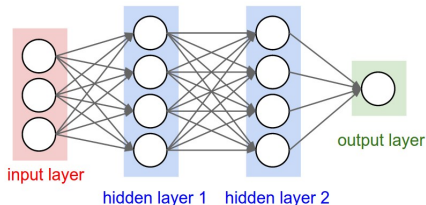
Section 2

Multilayer Perceptrons

An Artificial Neural Network (Multilayer Perceptron)

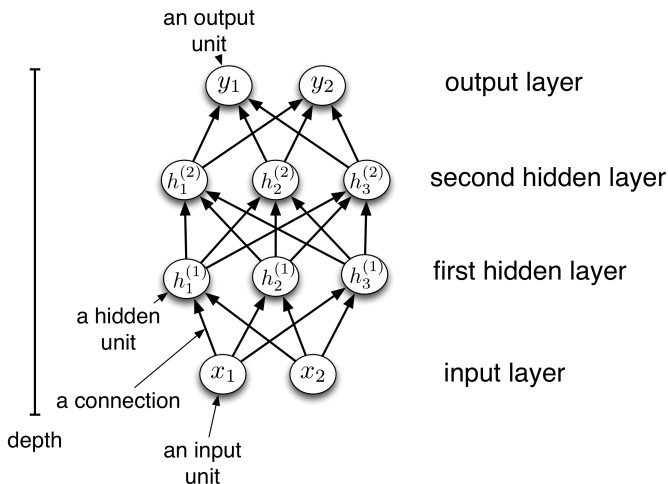
Idea:

- Use a simplified (mathematical) model of a neuron as building blocks
- Connect the neurons together into a directed acyclic graph in the following way:



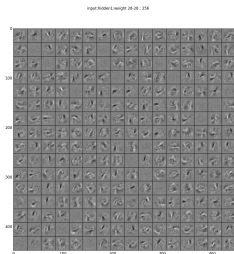
- An **input layer**: feed in input features (e.g. like retinal cells in your eyes)
- A number of **hidden layers**: don't have specific pre-defined meaning
- An **output layer**: interpretable output

Multilayer Perceptrons



This is a **three-layer neural network** with two hidden layers.

Example: MNIST Digit Recognition

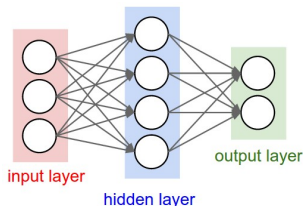


With logistic regression, we would have:

- Input: A 28x28 pixel image
 - ▶ $\mathbf{x}$ is a vector of length 784
- Target: The digit represented in the image
 - ▶ $\mathbf{t}$ is a one-hot vector of length 10
- Model
 - ▶ $\mathbf{y} = \text{softmax}(\mathbf{W}\mathbf{x} + \mathbf{b})$

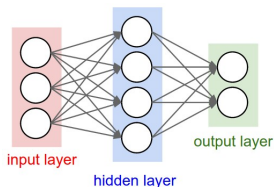
Example: MNIST Digit Recognition with a NN

Two layer neural network



- Input size: 784 (number of features)
- Hidden size: 50 —or another number that we choose as a *hyperparameter*
- Output size: 10 (number of classes)

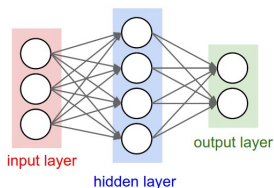
Making Predictions: computing the hidden layer



$$h_1 = f\left(\sum_{i=1}^{784} w_{1,i}^{(1)} x_i + b_1^{(1)}\right)$$
$$= f(x_1 w_{1,1}^{(1)} + x_2 w_{1,2}^{(1)} + \cdots + x_{784} w_{1,784}^{(1)} + b_1^{(1)})$$

$$h_2 = f\left(\sum_{i=1}^{784} w_{2,i}^{(1)} x_i + b_2^{(1)}\right)$$
$$= \dots$$

Making Predictions: computing the hidden layer



$$h_1 = f\left(\sum_{i=1}^{784} w_{1,i}^{(1)} x_i + b_1^{(1)}\right)$$

$$= f(x_1 w_{1,1}^{(1)} + x_2 w_{1,2}^{(1)} + \cdots + x_{784} w_{1,784}^{(1)} + b_1^{(1)})$$

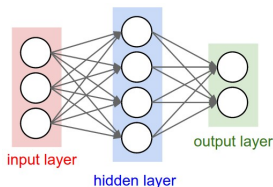
$$h_2 = f\left(\sum_{i=1}^{784} w_{2,i}^{(1)} x_i + b_2^{(1)}\right)$$

$= \dots$

Vectorizing: $h_1 = f((\mathbf{w}_{1,\cdot}^{(1)})^T \mathbf{x} + b_1^{(1)})$

Similar for $h_2, \dots, h_{50}$.

Making Predictions: computing the hidden layer



$$\begin{aligned}h_1 &= f\left(\sum_{i=1}^{784} w_{1,i}^{(1)} x_i + b_1^{(1)}\right) \\&= f\left(x_1 w_{1,1}^{(1)} + x_2 w_{1,2}^{(1)} + \cdots + x_{784} w_{1,784}^{(1)} + b_1^{(1)}\right)\end{aligned}$$

$$\begin{aligned}h_2 &= f\left(\sum_{i=1}^{784} w_{2,i}^{(1)} x_i + b_2^{(1)}\right) \\&= \dots\end{aligned}$$

Vectorizing: $h_1 = f((\mathbf{w}_{1,\cdot}^{(1)})^T \mathbf{x} + b_1^{(1)})$

Similar for $h_2, \dots, h_{50}$.

Thus, we have

$$\mathbf{h} = f(\mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)})$$

Making Predictions: computing the output (pre-activation)

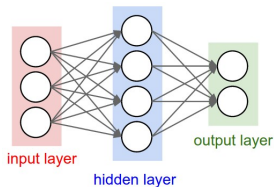
$$\mathbf{h} = f(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$z_1 = \sum_{j=1}^{50} w_{1,j}^{(2)} h_j + b_1^{(2)}$$

$$= w_{1,1}^{(2)} h_1 + w_{1,2}^{(2)} h_2 + \cdots + w_{1,50}^{(2)} h_{50} + b_1^{(2)}$$

$$z_2 = \sum_{j=1}^{50} w_{2,j}^{(2)} h_j + b_2^{(2)}$$

$$= \dots$$



Making Predictions: computing the output (pre-activation)

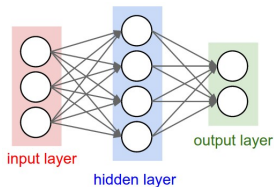
$$\mathbf{h} = f(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$z_1 = \sum_{j=1}^{50} w_{1,j}^{(2)} h_j + b_1^{(2)}$$

$$= w_{1,1}^{(2)} h_1 + w_{1,2}^{(2)} h_2 + \cdots + w_{1,50}^{(2)} h_{50} + b_1^{(2)}$$

$$z_2 = \sum_{j=1}^{50} w_{2,j}^{(2)} h_j + b_2^{(2)}$$

$$= \dots$$



Vectorizing: $z_1 = (\mathbf{w}_{1,\cdot}^{(2)})^T \mathbf{h} + b_1^{(2)}$

Similar for $z_2, \dots, z_{10}$.

Making Predictions: computing the output (pre-activation)

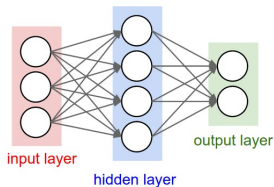
$$\mathbf{h} = f(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$z_1 = \sum_{j=1}^{50} w_{1,j}^{(2)} h_j + b_1^{(2)}$$

$$= w_{1,1}^{(2)} h_1 + w_{1,2}^{(2)} h_2 + \cdots + w_{1,50}^{(2)} h_{50} + b_1^{(2)}$$

$$z_2 = \sum_{j=1}^{50} w_{2,j}^{(2)} h_j + b_2^{(2)}$$

$$= \dots$$



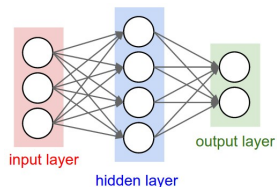
Vectorizing: $z_1 = (\mathbf{w}_{1,\cdot}^{(2)})^T \mathbf{h} + b_1^{(2)}$

Similar for $z_2, \dots, z_{10}$.

Thus, we have

$$\mathbf{z} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$$

Making Predictions: applying the output activation



$$\mathbf{h} = f(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\mathbf{z} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$$

$$\mathbf{y} = \text{softmax}(\mathbf{z})$$

Multi-Layer Perceptrons

More generally, we can have more than one hidden layer in a neural network.

$$\mathbf{h}^{(1)} = f(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\mathbf{h}^{(2)} = f(\mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)})$$

...

$$\mathbf{h}^{(L-1)} = f(\mathbf{W}^{(L-1)}\mathbf{h}^{(L-2)} + \mathbf{b}^{(L-1)})$$

$$\mathbf{z} = \mathbf{W}^{(L)}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)}$$

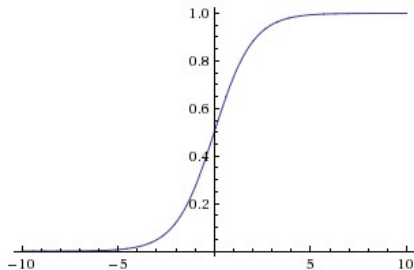
$$\mathbf{y} = \text{softmax}(\mathbf{z})$$

Activation Functions: common choices

Common Choices for f at hidden layers:

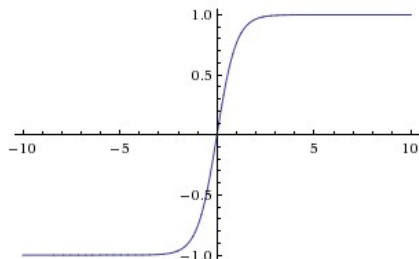
- Sigmoid activation
- Tanh activation
- ReLU activation

Activation Function: Sigmoid



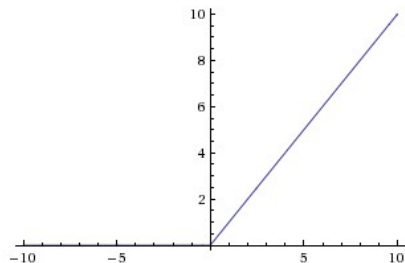
- somewhat problematic due to gradient signal at extreme input values
- all activations are positive

Activation Function: Tanh



- scaled version of the sigmoid activation
- also somewhat problematic due to gradient signal at extreme input values
- activations can be positive or negative

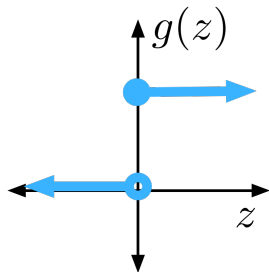
Activation Function: ReLU



- most often used nowadays
- all activations are positive
- easy to compute gradients
- can be problematic if the bias is too large and negative, so the activations are always 0

General rule: Start with ReLU activation. If necessary, try tanh.

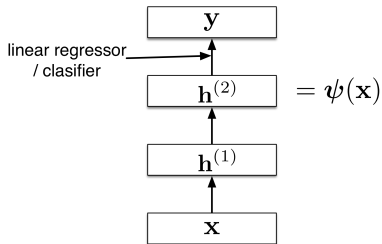
Activation Function: Hard Threshold



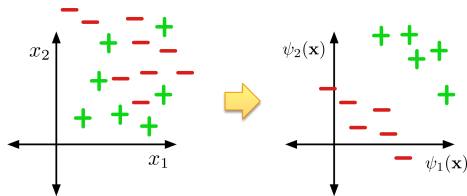
- can't be used in conjunction with gradient descent (why?)
- can be used in toy examples and when hand-picking parameters

Feature Learning

Neural nets can be viewed as a way of learning features:



The goal is for these features to become linearly separable:



Multi-Layer Perceptrons and Feature Learning

Think of computing $h^{(L-1)}$ as learning features:

$$\mathbf{h}^{(1)} = f(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

$$\mathbf{h}^{(2)} = f(\mathbf{W}^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)})$$

...

$$\mathbf{h}^{(L-1)} = f(\mathbf{W}^{(L-1)}\mathbf{h}^{(L-2)} + \mathbf{b}^{(L-1)})$$

And the final layer as a linear classification layer:

$$\mathbf{z} = \mathbf{W}^{(L)}\mathbf{h}^{(L-1)} + \mathbf{b}^{(L)}$$

$$\mathbf{y} = \text{softmax}(\mathbf{z})$$

Exercise: Neural Network Prediction Computation

Remember the NHANES heart disease prediction problem? We restrict ourselves to $d = 4$ features (age, BMI, systolic bp, diastolic bp):

$$\mathbf{x}^{(1)} = \begin{bmatrix} 62 \\ 29 \\ 104 \\ 56 \end{bmatrix}$$

Suppose we have a two layer neural network with

$$\mathbf{W}^{(1)} = \begin{bmatrix} 1 & 0 \\ 2 & 0 \\ 0.5 & 1 \\ 0.5 & -1 \end{bmatrix}^T, \quad \mathbf{b}^{(1)} = \begin{bmatrix} -150 \\ -50 \end{bmatrix}, \quad \mathbf{W}^{(2)} = \begin{bmatrix} 0.5 \\ 1 \end{bmatrix}^T, \quad \mathbf{b}^{(2)} = [-30]$$

With $\mathbf{h} = \text{ReLU}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$ and $\mathbf{y} = \sigma(\mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)})$. What is the prediction for the above data point?

Exercise: Neural Network Prediction Computation

$$\mathbf{W}^{(1)} = \begin{bmatrix} 1 & 0 \\ 2 & 0 \\ 0.5 & 1 \\ 0.5 & -1 \end{bmatrix}^T, \quad \mathbf{b}^{(1)} = \begin{bmatrix} -150 \\ -50 \end{bmatrix}, \quad \mathbf{W}^{(2)} = \begin{bmatrix} 0.5 \\ 1 \end{bmatrix}^T, \quad \mathbf{b}^{(2)} = \begin{bmatrix} -30 \end{bmatrix}$$

To compute $\mathbf{y}$ we compute:

$$\mathbf{h} = \text{ReLU}(\mathbf{W}^{(1)} \begin{bmatrix} 62 \\ 29 \\ 104 \\ 56 \end{bmatrix} + \mathbf{b}^{(1)})$$

$$= \text{ReLU}\left(\begin{bmatrix} 50 \\ -2 \end{bmatrix}\right) = \begin{bmatrix} 50 \\ 0 \end{bmatrix}$$

$$\mathbf{y} = \sigma(\mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)})$$

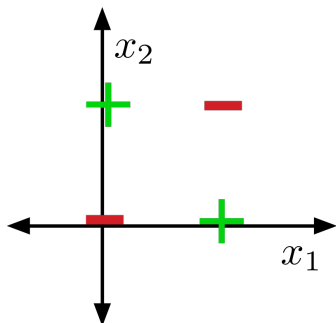
$$= \sigma(-5) \approx 0$$

Section 3

Expressive Power of Neural Networks

Hand Designing a Neural Network

Recall that we could not perfectly classify this data set using a linear model.

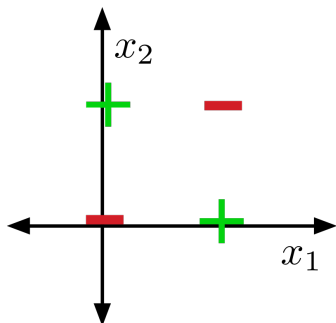


x_1	x_2	t
1	0	1
0	1	1
0	0	0
1	1	0

We will show that a neural network *can* perfectly classify this data set by hand-picking weights to a neural network.

Computing XOR

Example: Design a 2-layer neural network with 2 hidden units to compute XOR. Hand pick the weights/biases of this network.

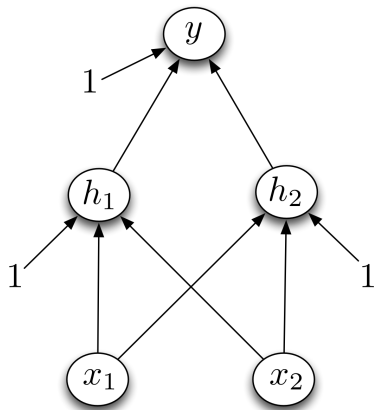


x_1	x_2	t
1	0	1
0	1	1
0	0	0
1	1	0

Use the hard threshold activation function for both hidden and final layers.

- $f(x) = 1$ if $x \geq 0$
- $f(x) = 0$ otherwise

Computing XOR: Approach



If we were to compute XOR by hand, we might compute

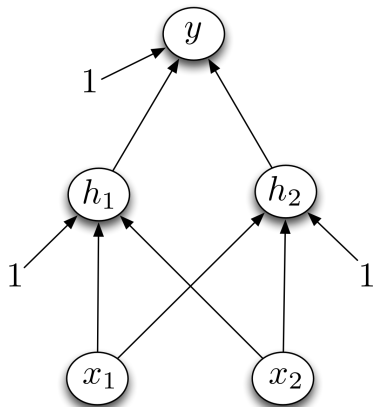
$$h_1 = \text{OR}(x_1, x_2)$$

$$h_2 = \text{AND}(x_1, x_2)$$

If we have these two features, then the problem is linearly separable:

$$y \approx h_1 - h_2$$

Computing XOR: Choosing Weights



Choose weights such that:

$$h_1 = 1 \text{ iff either } x_1 \text{ and } x_2 = 1 \\ = f(x_1 + x_2 - 0.5)$$

$$h_2 = 1 \text{ iff both } x_1 \text{ and } x_2 = 1 \\ = f(x_1 + x_2 - 1.5)$$

$$y \approx h_1 - h_2$$

$$y = f(h_1 - h_2 - 0.5)$$

Then we can fill in weights so that the network corresponds to these computations!

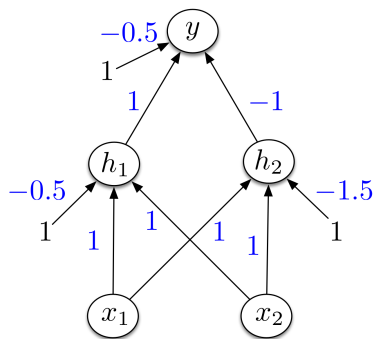
Computing XOR: Chosen weights

Choose weights such that:

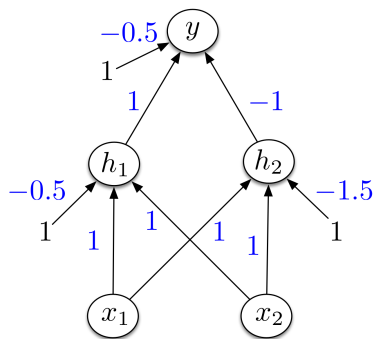
$$h_1 = f(x_1 + x_2 - 0.5)$$

$$h_2 = f(x_1 + x_2 - 1.5)$$

$$y = f(h_1 - h_2 - 0.5)$$



Computing XOR: Chosen weights



Choose weights such that:

$$h_1 = f(x_1 + x_2 - 0.5)$$

$$h_2 = f(x_1 + x_2 - 1.5)$$

$$y = f(h_1 - h_2 - 0.5)$$

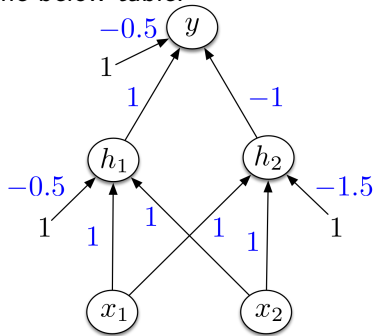
Vectorizing, we have:

$$\begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = f\left(\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \begin{bmatrix} -0.5 \\ -1.5 \end{bmatrix}\right)$$

$$y = f\left(\begin{bmatrix} 1 & -1 \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} + (-0.5)\right)$$

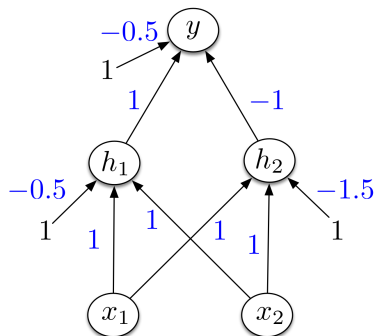
Computing XOR: making predictions

Exercise: What prediction will this network make for these four inputs? Fill in the below table:



x_1	x_2	h_1	h_2	y
1	0			
0	1			
0	0			
1	1			

Computing XOR: It works!



x_1	x_2	h_1	h_2	y
1	0	1	0	1
0	1	1	0	1
0	0	0	0	0
1	1	1	1	0

Computing XOR Demo

Demo: <https://playground.tensorflow.org/>

Expressive Power: Linear Layers (No Activation Function)

We have seen that there are some functions that linear classifiers can't represent. Are deep networks any better?

Not without activation functions!

Any sequence of *linear* layers (with no activation function) can be equivalently represented with a single linear layer.

$$\begin{aligned}\mathbf{y} &= \mathbf{W}^{(2)}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)} \\ &= \mathbf{W}'\mathbf{x} + \mathbf{b}'\end{aligned}$$

Deep linear networks with no activation functions are no more expressive than linear regression!

Expressive Power: MLP (nonlinear activation)

- Multilayer feed-forward neural nets with *nonlinear* activation functions are **universal approximators**: they can approximate any function arbitrarily well.
- This has been shown for various activation functions (thresholds, logistic, ReLU, etc.)
 - ▶ Even though ReLU is *almost* linear, it's nonlinear enough!

Universality for binary inputs and targets

Suppose we use hard threshold hidden units, and linear output

Strategy: 2^D hidden units, each of which responds to one particular input configuration

- This structure can learn any function.
- It only requires one hidden layer, though that layer needs to be extremely wide!

Universality for binary inputs and targets

Suppose we use hard threshold hidden units, and linear output

Strategy: 2^D hidden units, each of which responds to one particular input configuration

- This structure can learn any function.
- It only requires one hidden layer, though that layer needs to be extremely wide!

Limits of universality

- You may need to represent an exponentially large network.
- If you can learn any function, you might just overfit.

Section 4

Training Neural Networks: Backpropagation

Training Neural Networks

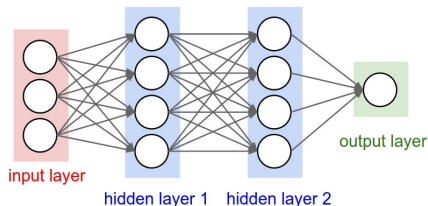
- How do we find good parameters (weights) for the neural network?
- We can continue to use the loss functions:
 - ▶ cross-entropy loss for classification
 - ▶ square loss for regression
- The neural network operations we used (addition, multiplication, activations) are continuous

We can use gradient descent!

- Start with a set of parameters (initialize to some value)
- Compute the derivative $\frac{\partial \mathcal{E}}{\partial w}$ for each scalar parameter w in the network
- Update the parameters towards the negative direction of the gradient

Gradient Descent for Neural Networks

Conceptually, the exact same idea as before. Take any parameter in this network:



The parameters affect the cost $\mathcal{E}$ in a differentiable way. Thus, we can compute $\frac{\partial \mathcal{E}}{\partial w}$ using standard calculus techniques!

BUT, we have more parameters than before

- Higher dimensional
- Harder to visualize
- More “steps”

Gradient Descent for Neural Networks: Plan

Since $\frac{\partial \mathcal{E}}{\partial w}$ is the average of $\frac{\partial \mathcal{L}}{\partial w}$ across training examples, we focus on computing $\frac{\partial \mathcal{L}}{\partial w}$

Plan:

- Use the univariate and multivariate chain rule
- Seeing how the same “steps” are repeated for gradient computation for different weights
- Drawing the **computation graph**

Let's review some examples that motivates the use of the chain rule.

Least Squares with a Logistic Model

Recall: univariate logistic least squares model

$$z = wx + b$$

$$y = \sigma(z)$$

$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

Let's compute the loss derivative.

Naive Gradient Computation

$$\begin{aligned}\mathcal{L} &= \frac{1}{2}(\sigma(wx + b) - t)^2 \\ \frac{\partial \mathcal{L}}{\partial w} &= \frac{\partial}{\partial w} \left[\frac{1}{2}(\sigma(wx + b) - t)^2 \right] \\ &= \frac{1}{2} \frac{\partial}{\partial w} (\sigma(wx + b) - t)^2 \\ &= (\sigma(wx + b) - t) \frac{\partial}{\partial w} (\sigma(wx + b) - t) \\ &= (\sigma(wx + b) - t) \sigma'(wx + b) \frac{\partial}{\partial w} (wx + b) \\ &= (\sigma(wx + b) - t) \sigma'(wx + b) x\end{aligned}$$

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial b} &= \frac{\partial}{\partial b} \left[\frac{1}{2}(\sigma(wx + b) - t)^2 \right] \\ &= \frac{1}{2} \frac{\partial}{\partial b} (\sigma(wx + b) - t)^2 \\ &= (\sigma(wx + b) - t) \frac{\partial}{\partial b} (\sigma(wx + b) - t) \\ &= (\sigma(wx + b) - t) \sigma'(wx + b) \frac{\partial}{\partial b} (wx + b) \\ &= (\sigma(wx + b) - t) \sigma'(wx + b)\end{aligned}$$

Do you notice any inefficiencies in this computation?

A More Structured Way to Compute the Derivatives

$$z = wx + b$$

$$y = \sigma(z)$$

$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

$$\frac{d\mathcal{L}}{dy} = y - t$$

$$\frac{d\mathcal{L}}{dz} = \frac{d\mathcal{L}}{dy} \sigma'(z)$$

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{d\mathcal{L}}{dz} x$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{d\mathcal{L}}{dz}$$

Less repeated work; easier to write a program to efficiently compute derivatives

Error Signal Notation

- Use $\bar{y}$ to denote the derivative $\frac{d\mathcal{L}}{dy}$, called the **error signal**
 - ▶ $\bar{y} = \frac{d\mathcal{L}}{dy}$
 - ▶ $\bar{z} = \frac{d\mathcal{L}}{dz}$
 - ▶ ...
- This notation emphasizes that the error signals are just *values* our program is computing (rather than a mathematical operation).
- This is notation introduced by Prof. Roger Grosse, and not standard notation

Computing predictions

$$z = wx + b$$

$$y = \sigma(z)$$

$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

Computing gradients

$$\bar{y} = y - t$$

$$\bar{z} = \bar{y}\sigma'(z)$$

$$\bar{w} = \bar{z}x$$

$$\bar{b} = \bar{z}$$

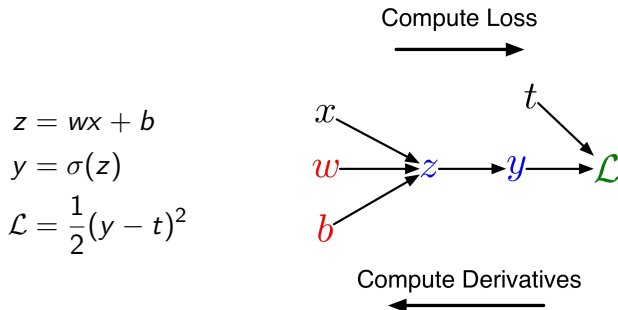
Neural Network Gradient Computation Approach

- 1 Draw a **computation graph**, making sure that the gradients we are interested in are captured in this computation graph
 - ▶ Final node: $\mathcal{L}$
 - ▶ Nodes of interest: the weights and biases
 - ▶ Intermediate nodes: hidden units
- 2 Compute the gradients of the loss with respect to the weights/biases, starting from the final node and working backwards.
- 3 Apply the multivariate chain rule to accumulate gradients along multiple paths.

We will also be able to vectorize our computation.

Computation Graph

We can draw out the steps involved in our prediction/loss computations using a *computation graph*.

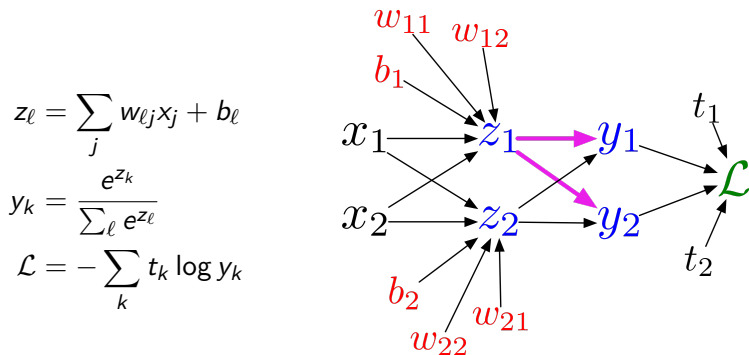


The *nodes* represent all the inputs and computed quantities

The *edges* represent which nodes are computed directly as a function of which other nodes.

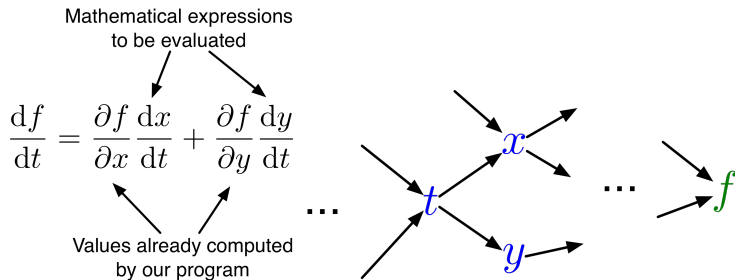
Multiclass Logistic Regression Computation Graph

The computation graph for a multiclass logistic regression problem with 2 features and 2 classes look like this:



- In general, the computation graph we work with *fans out* (i.e., each node depends on lots of other previous nodes)
- There are multiple paths for which a weight like w_{11} affects the loss $\mathcal{L}$.

Multivariate Chain Rule Notation



In our notation

$$\bar{t} = \bar{x} \frac{dx}{dt} + \bar{y} \frac{dy}{dt}$$

Drawing a Computation Graph

Exercise. Draw the computation graph for the following neural network, showing the relevant scalar quantities. Assume that $\mathbf{y}, \mathbf{h}, \mathbf{x} \in \mathbb{R}^2$

$$z_i = \sum_j w_{ij}^{(1)} x_j + b_i^{(1)}$$

$$h_i = \sigma(z_i)$$

$$y_k = \sum_i w_{ki}^{(2)} h_i + b_k^{(2)}$$

$$\mathcal{L} = \frac{1}{2} \sum_k (y_k - t_k)^2$$

Drawing a Computation Graph

Exercise. Draw the computation graph for the following neural network, showing the relevant scalar quantities. Assume that $\mathbf{y}, \mathbf{h}, \mathbf{x} \in \mathbb{R}^2$

$$z_1 = w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)}$$

$$z_2 = w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + b_2^{(1)}$$

$$h_1 = \sigma(z_1)$$

$$h_2 = \sigma(z_2)$$

$$y_1 = w_{11}^{(2)} h_1 + w_{12}^{(2)} h_2 + b_1^{(2)}$$

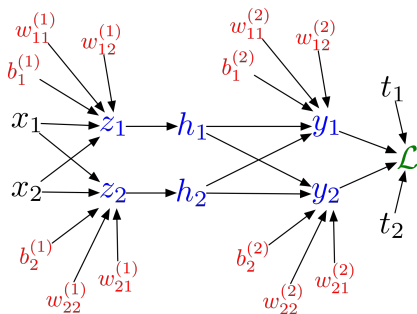
$$y_2 = w_{21}^{(2)} h_1 + w_{22}^{(2)} h_2 + b_2^{(2)}$$

$$\mathcal{L} = \frac{1}{2}((y_1 - t_1)^2 + (y_2 - t_2)^2)$$

Drawing a Computation Graph

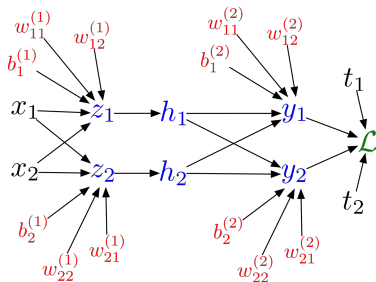
Exercise. Draw the computation graph for the following neural network, showing the relevant scalar quantities. Assume that $\mathbf{y}, \mathbf{h}, \mathbf{x} \in \mathbb{R}^2$

$$\begin{aligned}z_1 &= w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + b_1^{(1)} \\z_2 &= w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + b_2^{(1)} \\h_1 &= \sigma(z_1) \\h_2 &= \sigma(z_2) \\y_1 &= w_{11}^{(2)} h_1 + w_{12}^{(2)} h_2 + b_1^{(2)} \\y_2 &= w_{21}^{(2)} h_1 + w_{22}^{(2)} h_2 + b_2^{(2)} \\\mathcal{L} &= \frac{1}{2}((y_1 - t_1)^2 + (y_2 - t_2)^2)\end{aligned}$$



Efficient Gradient Computation Problem

What is $\frac{\partial \mathcal{L}}{\partial b_1^{(1)}}$? What is $\frac{\partial \mathcal{L}}{\partial w_{12}^{(1)}}$?

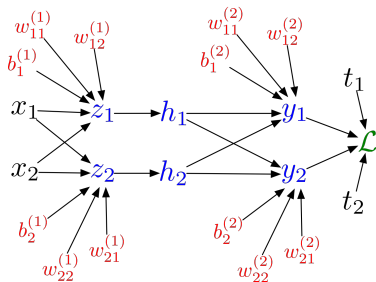


$$\frac{\partial \mathcal{L}}{\partial b_1^{(1)}} = \left(\frac{\partial \mathcal{L}}{\partial y_1} \frac{\partial y_1}{\partial h_1} + \frac{\partial \mathcal{L}}{\partial y_2} \frac{\partial y_2}{\partial h_1} \right) \frac{dh_1}{dz_1} \frac{\partial z_1}{\partial b_1^{(1)}}$$

$$\frac{\partial \mathcal{L}}{\partial w_{12}^{(1)}} = \left(\frac{\partial \mathcal{L}}{\partial y_1} \frac{\partial y_1}{\partial h_1} + \frac{\partial \mathcal{L}}{\partial y_2} \frac{\partial y_2}{\partial h_1} \right) \frac{dh_1}{dz_1} \frac{\partial z_1}{\partial w_{12}^{(1)}}$$

Efficient Gradient Computation Problem

What is $\frac{\partial \mathcal{L}}{\partial b_1^{(1)}}$? What is $\frac{\partial \mathcal{L}}{\partial w_{12}^{(1)}}$?



$$\frac{\partial \mathcal{L}}{\partial b_1^{(1)}} = \left(\frac{\partial \mathcal{L}}{\partial y_1} \frac{\partial y_1}{\partial h_1} + \frac{\partial \mathcal{L}}{\partial y_2} \frac{\partial y_2}{\partial h_1} \right) \frac{dh_1}{dz_1} \frac{\partial z_1}{\partial b_1^{(1)}}$$

$$\frac{\partial \mathcal{L}}{\partial w_{12}^{(1)}} = \left(\frac{\partial \mathcal{L}}{\partial y_1} \frac{\partial y_1}{\partial h_1} + \frac{\partial \mathcal{L}}{\partial y_2} \frac{\partial y_2}{\partial h_1} \right) \frac{dh_1}{dz_1} \frac{\partial z_1}{\partial w_{12}^{(1)}}$$

Lots of duplicate terms!

The Backpropagation Algorithm

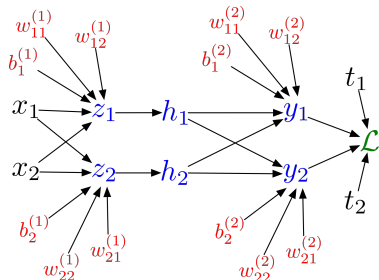
Backpropagation is an *algorithm* to compute gradients efficiently

- Forward Pass: Compute predictions (and save intermediate values)
- Backwards Pass: Compute gradients

The idea behind backpropagation is very similar to *dynamic programming*

- Use chain rule, and be careful about the order in which we compute the derivatives
- Compute derivatives **backwards**: i.e., starting from the nodes closest to $\mathcal{L}$, so that we do not need to recompute gradients!

Backpropagation for a MLP



Forward pass:

$$z_i = \sum_j w_{ij}^{(1)} x_j + b_i^{(1)}$$

$$h_i = \sigma(z_i)$$

$$y_k = \sum_i w_{ki}^{(2)} h_i + b_k^{(2)}$$

$$\mathcal{L} = \frac{1}{2} \sum_k (y_k - t_k)^2$$

Backward pass:

$$\bar{\mathcal{L}} = 1$$

$$\bar{y}_k =$$

$$\bar{w}_{ki}^{(2)} =$$

$$\bar{b}_k^{(2)} =$$

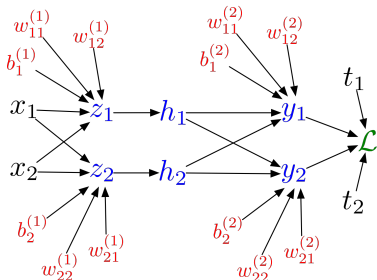
$$\bar{h}_i =$$

$$\bar{z}_i =$$

$$\bar{w}_{ij}^{(1)} =$$

$$\bar{b}_i^{(1)} =$$

Backpropagation for a MLP (Solution)



Forward pass:

$$z_i = \sum_j w_{ij}^{(1)} x_j + b_i^{(1)}$$

$$h_i = \sigma(z_i)$$

$$y_k = \sum_i w_{ki}^{(2)} h_i + b_k^{(2)}$$

$$\mathcal{L} = \frac{1}{2} \sum_k (y_k - t_k)^2$$

Backward pass:

$$\bar{\mathcal{L}} = 1$$

$$\bar{y}_k = \bar{\mathcal{L}}(y_k - t_k)$$

$$\bar{w}_{ki}^{(2)} = \bar{y}_k h_i$$

$$\bar{b}_k^{(2)} = \bar{y}_k$$

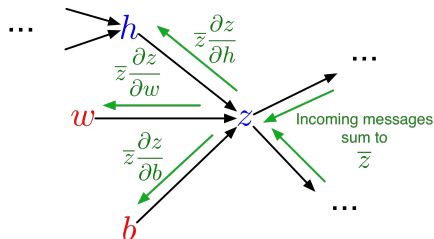
$$\bar{h}_i = \sum_k \bar{y}_k w_{ki}^{(2)}$$

$$\bar{z}_i = \bar{h}_i \sigma'(z_i)$$

$$\bar{w}_{ij}^{(1)} = \bar{z}_i x_j$$

$$\bar{b}_i^{(1)} = \bar{z}_i$$

Implementing Backpropagation



Forward pass: Each node...

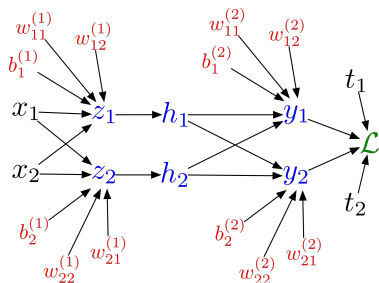
- receives messages (inputs) from its parents
- uses these messages to compute its own values

Backward pass: Each node...

- receives messages (error signals) from its children
- uses these messages to compute its own error signal
- passes message to its parents

This algorithm provides **modularity**!

Vectorizing MLP Forward Pass



$$z_i = \sum_j w_{ij}^{(1)} x_j + b_i^{(1)}$$

$$h_i = \sigma(z_i)$$

$$y_k = \sum_i w_{ki}^{(2)} h_i + b_k^{(2)}$$

$$\mathcal{L} = \frac{1}{2} \sum_k (y_k - t_k)^2$$

$$\mathbf{z} = \mathbf{W}^{(1)} \mathbf{x} + \mathbf{b}^{(1)}$$

$$\mathbf{h} = \sigma(\mathbf{z})$$

$$\mathbf{y} = \mathbf{W}^{(2)} \mathbf{h} + \mathbf{b}^{(2)}$$

$$\mathcal{L} = \frac{1}{2} \|\mathbf{y} - \mathbf{t}\|^2$$

Vectorizing MLP Backward Pass

$$\overline{\mathcal{L}} = 1$$

$$\overline{y_k} = \overline{\mathcal{L}}(y_k - t_k)$$

$$\overline{w_{ki}^{(2)}} = \overline{y_k} h_i$$

$$\overline{b_k^{(2)}} = \overline{y_k}$$

$$\overline{h_i} = \sum_k \overline{y_k} w_{ki}^{(2)}$$

$$\overline{z_i} = \overline{h_i} \sigma'(z_i)$$

$$\overline{w_{ij}^{(1)}} = \overline{z_i} x_j$$

$$\overline{b_i^{(1)}} = \overline{z_i}$$

$$\overline{\mathcal{L}} = 1$$

$$\overline{\mathbf{y}} =$$

$$\overline{\mathbf{W}^{(2)}} =$$

$$\overline{\mathbf{b}^{(2)}} =$$

$$\overline{\mathbf{h}} =$$

$$\overline{\mathbf{z}} =$$

$$\overline{\mathbf{W}^{(1)}} =$$

$$\overline{\mathbf{b}^{(1)}} =$$

Vectorizing MLP Backward Pass (Solution)

$$\overline{\mathcal{L}} = 1$$

$$\overline{y_k} = \overline{\mathcal{L}}(y_k - t_k)$$

$$\overline{w_{ki}^{(2)}} = \overline{y_k} h_i$$

$$\overline{b_k^{(2)}} = \overline{y_k}$$

$$\overline{h_i} = \sum_k \overline{y_k} w_{ki}^{(2)}$$

$$\overline{z_i} = \overline{h_i} \sigma'(z_i)$$

$$\overline{w_{ij}^{(1)}} = \overline{z_i} x_j$$

$$\overline{b_i^{(1)}} = \overline{z_i}$$

$$\overline{\mathcal{L}} = 1$$

$$\overline{\mathbf{y}} = \overline{\mathcal{L}}(\mathbf{y} - \mathbf{t})$$

$$\overline{\mathbf{W}^{(2)}} = \overline{\mathbf{y}} \mathbf{h}^T$$

$$\overline{\mathbf{b}^{(2)}} = \overline{\mathbf{y}}$$

$$\overline{\mathbf{h}} = \overline{\mathbf{W}^{(2)}}^T \overline{\mathbf{y}}$$

$$\overline{\mathbf{z}} = \overline{\mathbf{h}} \circ \sigma'(\mathbf{z}) = \text{diag}(\overline{\mathbf{h}}) \sigma'(\mathbf{z})$$

$$\overline{\mathbf{W}^{(1)}} = \overline{\mathbf{z}} \mathbf{x}^T$$

$$\overline{\mathbf{b}^{(1)}} = \overline{\mathbf{z}}$$

MLP Forward and Backward pass

Forward pass:

$$\mathbf{z} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

$$\mathbf{h} = \sigma(\mathbf{z})$$

$$\mathbf{y} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$$

$$\mathcal{L} = \frac{1}{2} \|\mathbf{y} - \mathbf{t}\|^2$$

Backward pass:

$$\bar{\mathcal{L}} = 1$$

$$\bar{\mathbf{y}} = \bar{\mathcal{L}}(\mathbf{y} - \mathbf{t})$$

$$\overline{\mathbf{W}^{(2)}} = \bar{\mathbf{y}}\mathbf{h}^T$$

$$\overline{\mathbf{b}^{(2)}} = \bar{\mathbf{y}}$$

$$\bar{\mathbf{h}} = \mathbf{W}^{(2)T} \bar{\mathbf{y}}$$

$$\bar{\mathbf{z}} = \bar{\mathbf{h}} \circ \sigma'(\mathbf{z}) = \text{diag}(\bar{\mathbf{h}}) \sigma'(\mathbf{z})$$

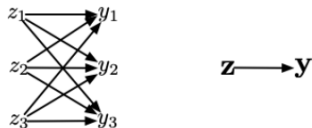
$$\overline{\mathbf{W}^{(1)}} = \bar{\mathbf{z}}\mathbf{x}^T$$

$$\overline{\mathbf{b}^{(1)}} = \bar{\mathbf{z}}$$

This computation is vectorized for a single data point. We can further vectorize this to compute predictions/cost over a batch of data. (You will do this in lab 6!)

Backpropagation in Vectorized Form

- Consider this computation graph:



- Backprop rules:

$$\mathbf{z} \in \mathcal{R}^N, \mathbf{y} \in \mathcal{R}^M \qquad \bar{z}_j = \sum_k \bar{y}_k \frac{\partial y_k}{\partial z_j} \qquad \bar{\mathbf{z}} = \frac{\partial \mathbf{y}}{\partial \mathbf{z}}^\top \bar{\mathbf{y}},$$

where $\partial \mathbf{y} / \partial \mathbf{z}$ is the **Jacobian matrix** (**note**: check the matrix shapes):

$$\left(\frac{\partial \mathbf{y}}{\partial \mathbf{z}} \right)_{M \times N} = \begin{pmatrix} \frac{\partial y_1}{\partial z_1} & \cdots & \frac{\partial y_1}{\partial z_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial z_1} & \cdots & \frac{\partial y_m}{\partial z_n} \end{pmatrix}$$

Backpropagation in practice

Backprop is used to train the overwhelming majority of neural nets today.

- Even optimization algorithms much fancier than gradient descent use backprop to compute the gradients.

Despite its practical success, backprop is believed to be neurally (biologically) implausible.

- No evidence for biological signals analogous to error derivatives.
- All the biologically plausible alternatives we know about learn much more slowly (on computers).
- So how on earth does the brain learn?